

API-based architectures is addressing these four basic questions:

- Are the services/APIs providing the functionality they were designed to deliver?
- Are the services/APIs secure?
- As a business do you understand the usage of APIs by users?
- Are the services/APIs providing the optimum performance to the user?

Visibility into dependencies

While addressing these questions, there are several problems that come up. Even as there are different solutions to these problems, there are simply too many gaps in them. There are several approaches to deal with and answer these questions. We can look into their dependencies. Microservices architecture, in general, is inherently very complex compared to the current distributed architecture that is being used. There is no end-to-end visibility into dependent services.

It is extremely important to understand the failure point. This is, in fact, the very first step of the debugging process because whenever something breaks we can only fix it if we have a good understanding of what is broken. It is difficult to decipher the connected metrics between services. So, in general, the visibility aspect is essential to this modern and complex architecture.

In order to find the problem areas, you need to correlate the multiple dashboards that are being used. Most developers are often trying to figure out this correlation between different services and metrics manually, which also increases the cognitive load on the development.

The role of logs

The metrics will tell you what broke, and the logs will tell you why it broke in the first place. The first logical step after looking at your metrics is to go to logs and understand why the failure happened that led everything to behave

abnormally. Query logs by requestID result in a huge number of logs in the microservices world. This is because if you have one call that is touching 1000 different services, then you have to deal with a huge pile of the log. And sequential navigation through logs to find out the root cause is hard. It is also hard to correlate due to the sheer amount of logs present.

Debugging slow queries

Sometimes your customer data does not load properly and your customers are unable to log in because the database having their data is slow. To deal with this, the traditional way is to manually log slow queries into the logging system. This also leads to an increased dependency on DBAs to get slow query logs.

Why we need observability

“Monitoring tells you whether a system is working; observability lets you ask why it isn’t working”
— *Baron Schwartz*

All questions about visibility, service interactions and slow queries can be answered by observability. In control theory, observability is defined as a measure of how well the internal states of a system can be inferred from the knowledge of that system’s external outputs. Simply put, observability is how well you can understand your complex system.

Need for observability

- The monoliths to microservices transition is happening and has happened in the past as well.
- Microservices create complex interactions.
- Failure modes are unpredictable these days.
- There is a rise in the API ecosystem.
- Monitoring can no longer help us.

Ability to answer abstract questions

If we are placing an order on Amazon and something goes wrong, then what characteristics did the queries that timed out at 500ms common — service versions, browser plugins?

Instrumentation produces data and querying the data answers this question. You may be wondering what this instrumentation data is. Well, there are three data sources that we are talking about here — metrics, logs and distributed tracing.

Metrics are basically the aggregated summary statistics. These are valuable because your operating system and applications generate metrics that are going to contain a name or a time stamp and a field that represents some value. They are the first obvious place to start with when it comes to monitoring. One issue that metrics present are that they lack context. You can see something weird going on with your system but you will never know what, unless you look into the second pillar — logs.

Logs contain detailed debugging information emitted by processes. They play perhaps the second most important role in your toolbox because they tend to give us more in-depth information about resources than metrics. So, if metrics show us that a resource has died, logs will let us know why it died in the first place. The problem with logs is that they are hard to correlate and bring into context. This brings us to our next pillar — distributed tracing.

Distributed tracing provides insights into the full life cycle, aka traces of requests to a system, allowing you to pinpoint failure and performance issues. It adds critical visibility into the end-to-end health of the application. For instance, whenever a request comes to the front-end, a unique ID is assigned to it and that request gets tracked throughout the system. Along the way, we collect telemetry data, which can be shown in different formats.